



华中科技大学 2024~2025 第一学期

“ 计算机系统基础 ” 考试试卷 (A 卷)

考试方式 闭卷 考试时间 2024-11-26 晚上 考试时长 150 分钟

院 (系) 专业班级

学 号 姓 名

题号	一	二	三	四	五			总分	总分人	核对人
分值	20	20	20	20	20			100		
得分										

分 数	
评卷人	

一、(20 分) 数据的表示和访问。

1. 有一个 C 语言程序如下。(10 分)

```
typedef struct{
    char    id[10];
    int     age;
    float    height;
} person_info;
void main() {
    person_info p1;
    strcpy(p1.id, "12350"); #字符 0 的 ASCII 为 0x30
    p1.age = 15;
    p1.height = 1.75f;
    unsigned short x1 = *(unsigned short*)(p1.id + 1);
    int y1 = -100;          -----①
}
```

将上述程序编译链接生成可执行程序,用调试工具运行完语句①后,观察变量地址如下: p1 的地址是 0xffffd038, p1.age 的地址是 0xffffd044, p1.height 的地址是 0xffffd048, x1 的地址是 0xffffd032, y1 的地址是 0xffffd034。以字节形式观察这些变量在内存中的存放形式。以 16 进制的形式填写程序用到的内存单元内容(无关的字节中的内容填写 XX, person_info 的大小为 20 字节)。

0xffffd032: _____

0xffffd03a: _____

0xffffd042: _____

0xffffd04a: _____。

提示: 浮点数 float 的编码规则: float 用 32 位表示, 最高位(第 31 位)为符号位, 23-30 位为用移码表示的指数(移码值为 0x7F), 0-22 位为尾数位。

2. 调试一个 C 程序时，看到源程序语句与对应的反汇编语句如下。根据观察到的信息填空。（10 分，每空 1 分）

```
int    arr1[5];
short  arr2[3][2];
void   main() {
    int i, j;
    i = 2;    // 执行该语句时，ebp 的值为 0xffffd068，eax 的值为 0x56558fdc
0x5655619d <+16>:  c7 45 ec 02 00 00 00    movl    $0x2, -0x14(%ebp)
    变量 i 的地址是 0x_____，该变量存放在_____段中。
    arr1[i] = 34;
0x565561a4 <+23>:  8b 55 ec    mov    -0x14(%ebp), %edx
0x565561a7 <+26>:  c7 84 90 30 00 00 00 22 00 00 00    movl    $0x22, 0x30(%eax,%edx,4)
    此时，%edx 存放的是_____的值，arr1[i]的地址是 0x_____。
    i = 1;
0x565561b2 <+37>:  movl    $0x1, -0x14(%ebp)
    j = 1;
0x565561b9 <+44>:  movl    $0x1, -0x10(%ebp)
    arr2[i][j] = 27;
0x565561c0 <+51>:  8b 55 ec    mov    -0x14(%ebp), %edx
0x565561c3 <+54>:  8d 0c 12    lea    (%edx,%edx,1), %ecx
    执行这条指令后，%ecx 中的值是_____，arr2 的每行有两个元素，%ecx 表示的含
    义是_____。
0x565561c6 <+57>:  8b 55 f0    mov    -0x10(%ebp), %edx
0x565561c9 <+60>:  01 ca    add    %ecx, %edx
0x565561cb <+62>:  66 c7 84 50 44 00 00 00 1b 00    movw    $0x1b, 0x44(%eax,%edx,2)
    arr2[i][j]的地址是 0x_____。
    int *p1 = &arr1[2];
0x565561d5 <+72>:  8d 90 30 00 00 00    lea    _____(%eax), %edx
0x565561db <+78>:  89 55 f4    mov    %edx, -0xc(%ebp)
    *(p1 + 1) = 20;
0x565561de <+81>:  mov    -0xc(%ebp), %edx
0x565561e1 <+84>:  add    _____, %edx
0x565561e4 <+87>:  movl    $0x14, (%edx)
    short *p3 = &arr2[1][1];
0x565561e7 <+90>:  8d 80 4a 00 00 00    lea    _____(%eax), %eax
0x565561ed <+96>:  89 45 fc    mov    %eax, -0x4(%ebp)
}
```

分 数	
评卷人	

二、(20 分) C 程序的转换

1. 阅读下面的程序段，回答问题（10 分，每空 1 分）

```
void main( ) {
    short s = -5;
    unsigned short us = 0xffffb;
    int i = s;                // ①
    unsigned int ui = us;     // ②
    if (ui < 5)                // ③
        s = (int)(s + s) >> 1;
    else
        us = (unsigned int)(us + us) >> 1;    // ④
}
```

变量 s、us、i、ui，对应的地址表达式为：-0xc(%ebp)、-0xa(%ebp)、-0x8(%ebp)、-0x4(%ebp)。

(1) 执行完①处语句后，i = 0x_____；

该语句对 s 变量的处理指令为：_____ -0xc(%ebp),%eax。

(2) 将②处的运算指令序列补充完整：

0x565561b0 <+35>: _____ -0xa(%ebp),%eax

0x565561b4 <+39>: mov _____ %eax,-0x4(%ebp)

执行完②处语句后 ui = _____；

(3) 对于③处语句，补充完整这条语句的处理序列：

0x565561b7 <+42>: cmpl \$0x5, -0x4(%ebp)

0x565561bb <+46>: _____ 0x565561cb <main+62>

③ 处 if-else 语句，执行分支_____。

(4) ④处语句对应的机器指令是：

0x565561cb <+62>: _____ -0xa(%ebp),%eax

0x565561cf <+66>: _____

0x565561d1 <+68>: _____ %eax

0x565561d3 <+70>: mov _____, -0xa(%ebp)

2. 阅读如下程序及其反汇编代码，回答问题（10 分）。

```
int max(int a, int b) {
    if (a > b) return a;
    else return b;
}

void main() {
    int x = 100;
    int y = -10;
    int m = max(x, y);
}
```

main 函数中，int m=max(x, y) 对应的反汇编代码如下：

```
0x565561ca <+30>:    push    -0x8(%ebp)          ----- ①
0x565561cd <+33>:    push    -0xc(%ebp)
0x565561d0 <+36>:    call    0x5655618d <max>
0x565561d5 <+41>:    add     $0x8,%esp          ----- ②
0x565561d8 <+44>:    mov     %eax, -0x4(%ebp)
```

max 函数的反汇编代码如下：

```
0x5655618d <+0>:    push    %ebp                ----- ③
0x5655618e <+1>:    mov     %esp, %ebp
0x5655619a <+13>:   mov     0x8(%ebp), %eax
0x5655619d <+16>:   cmp     0xc(%ebp), %eax
0x565561a0 <+19>:   jle     0x565561a7 <max+26>
0x565561a2 <+21>:   mov     0x8(%ebp), %eax
0x565561a5 <+24>:   jmp     0x565561aa <max+29>
0x565561a7 <+26>:   mov     0xc(%ebp), %eax
0x565561aa <+29>:   mov     %ebp, %esp
0x565561ac <+31>:   pop     %ebp                ----- ④
0x565561ad <+32>:   ret
```

设执行①处语句前，(esp)= 0xffffd058，(ebp)= 0xffffd068。

填写执行指令 ③ 后堆栈的示意图，包括 main 函数的局部变量 x, y 的位置和内容，无关的位置填写 XX。

指令④的作用是_____。若缺少指令④，子程序（能/不能）_____正常返回。

执行指令②前，esp=_____。指令②的作用是_____。

	0xffffd048
	0xffffd04c
	0xffffd050
	0xffffd054
	0xffffd058
	0xffffd05c
	0xffffd060
	0xffffd064
	0xffffd068

分 数	
评卷人	

三、(20 分) 程序的运行的基本原理

1. C 程序的部分反汇编代码如下。完成填空（10 分，每空 1 分）

```
void main() {
    int x = -1;
    0x565561b9 <+28>:    movl    $0xffffffff,-0x14(%ebp)
    int y = -2;
    0x565561c0 <+35>:    movl    $0xffffffe,-0x10(%ebp)
    int z = y - x;
    0x565561c7 <+42>:    mov     -0x10(%ebp),%edx
    0x565561ca <+45>:    sub     -0x14(%ebp),%edx          ----- ①
    0x565561cd <+48>:    mov     %edx,-0xc(%ebp)
    if (y == z)
```

```

0x565561d0 <+51>:    mov     -0x10(%ebp),%edx
0x565561d3 <+54>:    cmp     %edx,-0xc(%ebp)
0x565561d6 <+57>:    jne     0x565561ee <main+81> -----②
    printf("equal\n");
0x565561d8 <+59>:    sub     $0xc,%esp
    .....
0x565561ec <+79>:    jmp     0x56556202 <main+101> ----- ③
    else printf("not equal\n");
0x565561ee <+81>:    .....
0x56556202<+101>    .....
}

```

内存地址 0x565561c0—0x565561c6 的内存单元中，存放的是汇编语句 `mov $0xffffffff, 0x10(%ebp)` 的_____。

① 处的指令执行后，CF=____, OF=____, ZF=____, SF=_____。

程序在运行时，寄存器 EIP 中存放的是_____。当②处的指令预取和解码后，EIP 的值为_____。由于此时 ZF=_____，转移条件成立，②处的指令执行后，EIP 的值为_____。

③ 处的 `jmp` 指令，其在实现 `if-else` 语句时的作用是_____。

2. 已知函数 `swap` 的 C 语言框架如下。回答问题（10 分）

```

int a = 200, b = 100;
void swap(int *x, int *y) {
    0x5655618d <+0>:    push    %ebp
    0x5655618e <+1>:    mov     %esp,%ebp
    0x56556190 <+3>:    sub     $0x10,%esp
    0x56556193 <+6>:    call    0x565561ee <__x86.get_pc_thunk.ax>
    0x56556198 <+11>:   add     $0x2e44,%eax
    int tmp;
    if (*x > *y) {
        0x5655619d <+16>:   mov     0x8(%ebp),%eax
        0x565561a0 <+19>:   mov     (%eax),%edx
        0x565561a2 <+21>:   mov     0xc(%ebp),%eax
        0x565561a5 <+24>:   mov     (%eax),%eax
        0x565561a7 <+26>:   cmp     %eax,%edx
        0x565561a9 <+28>:   jle     0x565561c5 <swap+56>
        tmp = *x;
        *x = *y;
        *y = tmp;
    }
}
void main() {
    swap(&a, &b);
=> 0x565561d5 <+13>:   lea     0x30(%eax), %edx

```

```

0x565561db <+19>:    push    %edx
0x565561dc <+20>:    lea     0x2c(%eax), %eax
0x565561e2 <+26>:    push    %eax
0x565561e3 <+27>:    call   0x5655618d <swap>
(0x565561e3 <+27>:    e8 a5 ff ff ff  call   0x5655618d <swap>)
0x565561e8 <+32>:    add     $0x8, %esp
}

```

(1) 根据反汇编指令， swap(&a, &b)函数调用时，如何传递参数的地址？（2分）

(2) 在 swap 函数中，如何根据地址，获取 a, b 的值？（2分）

(3) 反汇编指令：call 0x5655618d <swap>，对应的机器指令为：e8 a5 ff ff ff，其中 a5 ff ff ff 的含义是什么？（3分）。

(4) 请写出 0x565561c5<swap+56> 处的机器指令，使得程序能正常返回。（3分）

分 数	
评卷人	

四、（20 分）程序分析与优化

1、根据反汇编代码，填补 C 语言语句缺失的部分（7 分，每空 1 分）

```

void main() {
    char str1[13] = _____;
    0x565561d5 <+40>:    movl    $0x64636261, -0x26(%ebp)    #字符 a 的 ASCII 为 0x61
    0x565561dc <+47>:    movl    $0x68676665, -0x22(%ebp)
    0x565561e3 <+54>:    movl    $0x6c6b6a69, -0x1e(%ebp)
    0x565561ea <+61>:    movb    $0x0, -0x1a(%ebp)

    char str2[13];
    for (int i = _____; _____; _____) {
    0x565561ee <+65>:    movl    $0x0, -0x2c(%ebp)
    0x565561f5 <+72>:    jmp     0x56556215 <main+104>
        _____ = _____;
    0x565561f7 <+74>:    lea     -0x26(%ebp), %ecx
    0x565561fa <+77>:    mov     -0x2c(%ebp), %edx

```

```

0x565561fd <+80>:  add    %ecx, %edx
0x565561ff <+82>:  movzbl (%edx), %edx
0x56556202 <+85>:  mov    %edx, %ecx
0x56556204 <+87>:  and    $0xf, %ecx
0x56556207 <+90>:  lea    -0x19(%ebp), %ebx    # -0x19(%ebp)代表 str2 数组首字符
0x5655620a <+93>:  mov    -0x2c(%ebp), %edx
0x5655620d <+96>:  add    %ebx, %edx
0x5655620f <+98>:  mov    %cl, (%edx)
    }
0x56556211 <+100>: addl    $0x1, -0x2c(%ebp)
0x56556215 <+104>: cmpl    $0xb, -0x2c(%ebp)
0x56556219 <+108>: jle     0x565561f7 <main+74>
    _____;
0x5655621b <+110>: movb    $0x0, -0xd(%ebp)
    }

```

2. 上述程序的功能是什么？（2 分）

3. 上述反汇编程序，效率较低的原因有哪些？（3 分）

4、常见的程序优化方法有哪些？（3 分）

5. 请用汇编语言程序片段给出功能等价的优化方案。（5 分）

分 数	
评卷人	

五、（20 分）程序的链接和异常控制流

1、有两个源文件: `main.c` 和 `test.c`，它们的内容如下，将这两个文件分别编译后再链接成一个执行程序。回答下列问题。（10 分）

```

/*main.c*/
int mul();
int a[5]={1,2,3,4,5};
int main() {
    int val;
    val=mul();
    printf("sum_val=%d\n", val);
    return 0;
}

```

```

/*test.c*/
extern int a[];
static int b[5]={6,7,8,9,10};
int mul() {
    int i, sum_val=0;
    for (i=0; i<5; i++)
    {
        sum_val = sum_val + a[i] * b[i];
    }
    return sum_val;
}

```

(1) 可重定位目标文件中有哪些节？每个节中包含有哪些信息？至少写出 4 个。(4 分)

(2) 对于编译生成的可重定位目标文件 main.o 和 test.o，各自生成了哪些符号？并说明其类型（全局、外部、本地），以及这些符号出现在各自模块的哪个节中。(4 分)

(3) main.c 中哪些语句中的那些符号需要重定位？(2 分)

2. 回答与中断和异常相关的问题 (10 分)

(1) 可屏蔽中断和不可屏蔽中断有什么区别？(2 分)

(2) 举 3 个异常的例子。(3 分)

(3) 用户单击了一下鼠标，请描述计算机的处理过程。(3 分)

(4) 中断向量表在中断响应过程中起到什么作用？(2 分)